

架构决策记录 ADR

团队：暴风星云裂 | 项目：CampusHub | 日期：2026年4月29日

ADR-001：采用前后端分离的分层单体架构

状态

已接受

背景

CampusHub 是 3 人学生团队在 10 周内完成的校园互助 MVP。系统需要覆盖注册登录、需求发布、接单订单、订单内文字和图片聊天、评价信用、举报审核和后台管理。P1 已明确不追求商业级复杂功能；P2 阶段将聊天范围限定为订单双方的文字和图片消息，不实现语音/视频通话、群聊、位置共享等高复杂度功能。

决策

我们决定采用 **前后端分离的分层单体架构**：前端使用 Vue3，后端使用一个 Spring Boot 应用，后端内部按 Controller、Service、Mapper/Repository 分层，并按业务模块组织代码。

理由

- 与既定技术栈 Vue3 + Spring Boot + MySQL 一致。
- 能在 10 周内集中完成业务闭环，减少基础设施成本。
- 单体应用更容易保障订单状态流转和信用分更新的一致性。
- 模块化包结构可以保留后续拆分服务的空间。

后果

- 正面影响：开发、部署、测试和调试成本低，适合课程项目。
- 负面影响：所有后端模块运行在同一进程内，无法按模块独立扩容。
- 需要关注的风险：如果模块边界不清晰，单体也会变成难维护的“大泥球”，因此必须约束模块间调用方式。

AI 辅助记录

- AI 初稿内容摘要：AI 在直接询问时同时推荐前后端分离、微服务、Redis、消息队列等通用方案。
- 人工修订内容：去掉微服务和中间件建议，明确采用分层单体，并强调模块化边界。
- 修订理由：AI 初稿偏向成熟商业平台模板，没有充分考虑 3 人团队、10 周交付和无需中间件的约束。

AI 初稿与人工修订 diff:

- 可以采用微服务架构，将用户、订单、消息、评价等拆成独立服务，并使用 Redis 缓存和消息队列处理通知。
- + 采用前后端分离的分层单体架构，后端保持一个 Spring Boot 应用，内部按业务模块划分。
- + MVP 阶段不引入 Redis、消息队列、网关等中间件，通过 MySQL 事务保障核心流程一致性。

ADR-002：后端采用 Spring Boot + RESTful API

状态

已接受

背景

P1 需求要求系统支持主流浏览器访问、前后端分离、RESTful API 和 JSON 数据格式。团队希望采用主流 Java 技术栈，便于资料查找、代码生成、接口测试和后续维护。

决策

后端采用 **Java 17 + Spring Boot + Spring MVC** 构建 RESTful API，使用 Spring Security + JWT 完成认证授权，使用 Bean Validation 做参数校验。

理由

- Spring Boot 是主流 Java Web 框架，生态成熟。
- Spring MVC 与 RESTful API 配合稳定，便于前后端并行开发。
- Spring Security 能覆盖登录鉴权、角色权限、接口保护等需求。
- JWT 适合前后端分离场景，避免后端保存会话。

后果

- 正面影响：后端结构清晰，团队可以快速开发用户、需求、订单等模块。
- 负面影响：Spring Security 配置有一定学习成本。
- 需要关注的风险：JWT 过期、刷新、退出登录和管理员权限必须统一设计，不能各接口自行判断。

AI 辅助记录

- AI 初稿内容摘要：AI 建议后端可以选择 Spring Boot、Django、Express 等任一主流框架。
- 人工修订内容：固定为 Java 17 + Spring Boot，并补充 Spring Security + JWT、Bean Validation。
- 修订理由：用户已指定后端 Java 和 Spring Boot；架构文档应给出明确选择，而不是开放多个候选。

AI 初稿与人工修订 diff：

- 后端可以选择 **Spring Boot、Django、Express** 等主流框架，接口形式可根据团队熟悉程度决定。
- + 后端固定采用 **Java 17 + Spring Boot + Spring MVC**，接口统一采用 **RESTful JSON**。
- + 认证授权统一使用 **Spring Security + JWT**，参数校验使用 **Bean Validation**。

ADR-003：数据库采用 MySQL 8.0 单库，不引入缓存和消息队列

状态

已接受

背景

CampusHub 的数据主要是结构化业务数据，包括用户、需求、订单、聊天消息、评价、举报、公告和操作日志。P1 非功能需求提出支持约 200 名同时在线用户，核心查询分页展示，每页最多 20 条记录。用户明确要求数据库选择 MySQL，学生项目不需要中间件。

决策

数据库采用 **MySQL 8.0 单库**。MVP 阶段不引入 Redis、RabbitMQ、Kafka、Elasticsearch 等中间件。订单内文字和图片消息持久化到 MySQL，并由 Spring WebSocket 在单体应用内实时推送；系统通知通过 MySQL 通知表实现，前端轮询未读数；搜索通过 MySQL 索引和 LIKE/FULLTEXT 逐步实现；定时任务使用 Spring Task。

理由

- MySQL 支持事务，适合订单状态和信用分等一致性要求。
- 当前数据量和并发目标不需要缓存、消息队列或独立搜索引擎。
- 单库部署最简单，降低本地开发、CI 和云服务器部署难度。
- 不引入中间件可以减少故障点和运维学习成本。

后果

- 正面影响：部署简单，事务一致性清晰，便于数据备份和调试。
- 负面影响：热点查询和全文搜索能力有限。
- 需要关注的风险：必须设计合理索引、分页查询和状态字段，避免列表接口全表扫描。

AI 辅助记录

- AI 初稿内容摘要：AI 建议 MySQL + Redis + RabbitMQ + Elasticsearch，以支持缓存、异步通知和搜索。
- 人工修订内容：保留 MySQL，删除 Redis、RabbitMQ、Elasticsearch；订单内聊天使用 Spring WebSocket + MySQL 消息表，系统通知使用数据库通知表和轮询，搜索使用 MySQL 索引和 Spring Task。
- 修订理由：AI 初稿没有区分“未来可能需要”和“MVP 必须需要”，引入过多中间件会使学生项目偏离核心业务。

AI 初稿与人工修订 diff：

- 使用 MySQL 存储业务数据，Redis 缓存热门需求列表，RabbitMQ 异步发送通知，Elasticsearch 支持任务搜索。
+ 使用 MySQL 8.0 单库存储业务数据。
+ 不采用 Redis、RabbitMQ/Kafka、Elasticsearch。
+ 订单内文字和图片聊天写入 order_message 表并通过 Spring WebSocket 推送；系统通知写入 notification 表并由前端轮询；搜索先使用 MySQL 索引和 LIKE/FULLTEXT；过期任务使用 Spring Task。

ADR-004：前端采用 Vue3 技术栈并复用同一工程承载前台和后台

状态

已接受

背景

CampusHub 需要学生前台和管理员后台。前台包括任务大厅、发布需求、订单详情、消息中心、个人资料等页面；后台包括用户管理、需求管理、订单管理、举报处理、公告管理等页面。后台功能较轻，不需要独立管理系统工程。

决策

前端采用 **Vue3 + TypeScript + Vite + Vue Router + Pinia + Element Plus + Axios**。前台和后台共用一个 Vue3 工程，通过路由前缀、角色权限和布局组件区分。

理由

- Vue3 与课程/学生项目常见技术栈匹配，学习成本低。
- Element Plus 表单、表格、弹窗组件成熟，适合后台管理页面。
- 单工程可以复用登录态、API 封装、文件上传、权限守卫和通用组件。
- 前后台分开工程会增加构建、部署和权限同步成本。

后果

- 正面影响：前端开发效率高，重复代码少，部署简单。
- 负面影响：工程内页面较多，需要清晰目录结构避免混乱。
- 需要关注的风险：后台路由必须有前端守卫和后端接口双重权限校验，不能只依赖前端隐藏菜单。

AI 辅助记录

- AI 初稿内容摘要：AI 建议前台 Vue3，后台可以使用单独的 Admin 模板工程。
- 人工修订内容：改为单个 Vue3 工程承载前台和后台，使用路由和权限区分。
- 修订理由：后台功能只是基础管理，拆成两个前端工程会增加学生团队维护成本。

AI 初稿与人工修订 diff:

- 前端用户端使用 **Vue3**，管理员端可以单独使用一个后台管理模板工程。
- + 前台和后台共用一个 **Vue3** 工程。
- + 使用 **Vue Router** 路由前缀、**Pinia** 登录态和后端角色权限控制区分普通学生与管理员。